Vision2Action: reproducible OpenCV 5 perception evaluation for a mobile manipulator

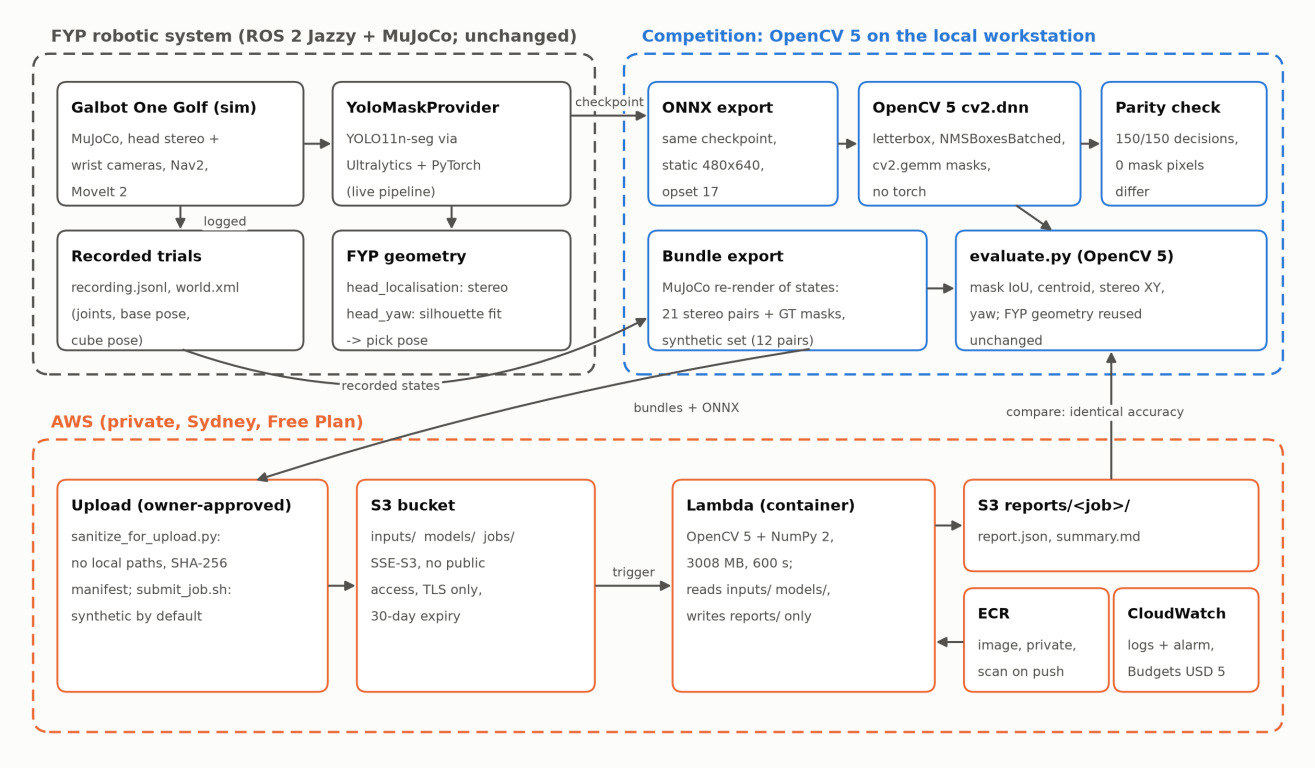
Technical report, OpenCV AI Competition 2026. All numbers were measured on 2026-10-22 and come from the committed reports in `results/`. Figures are in `docs/figures/`.

Summary

Vision2Action takes the perception stage of an existing ROS 2 mobile manipulation system (a Galbot One Golf humanoid in MuJoCo, built as a final-year project) and makes it **portable, checkable and reproducible with OpenCV 5**:

1. **OpenCV 5 inference.** The project's YOLO11n-seg cube detector runs in OpenCV 5 `cv2.dnn`, with no PyTorch or Ultralytics. On 150 validation images it gives the **same detection decisions (150/150) and identical masks (0 differing pixels)** as the original Ultralytics pipeline, and it is faster on the same CPU: 20.9 ms against 25.3 ms median end to end.
2. **A perception evaluation framework.** Built on OpenCV 5 and the robot's own geometry code. It scores segmentation, stereo position and yaw against simulator ground truth on re-rendered robot trials, recorded missions and a synthetic set.
3. **A reproducible cloud evaluation.** The same evaluation runs as a container-image AWS Lambda (S3 → Lambda → S3) in a private, Free Plan project. Accuracy on AWS is **identical to the local run**, with differences below 1e-12. Latency differs and is reported separately.
4. **OpenCV 5 on the live robot.** With `yolo_backend:=opencv5`, every camera frame the robot uses during a pick-and-place mission is segmented by OpenCV 5. All **5 of 5** recorded live missions succeeded, with final placement errors of **3.3–6.1 mm**.

The robot's navigation, manipulation, mission and recovery code was not modified. The only robot-side change is an opt-in detector backend; the default is still Ultralytics.



1. System context

The robot runs ROS 2 Jazzy with Nav2 and MoveIt 2 on a MuJoCo model of the Galbot One Golf. It drives to a table, finds a 30×30×60 mm red block and picks it with one arm. The perception pipeline it uses for the pick:

- a fine-tuned YOLO11n-seg model segments the block in the head stereo cameras (`YoloMaskProvider`, Ultralytics on PyTorch)
- `head_localisation` triangulates the mask centroids into a 3D point
- `head_yaw` fits the block's yaw with a coarse-to-fine silhouette IoU search over a box model

Each mission is recorded (joints, base pose, block pose, mission events) to `recording.jsonl`. The recordings hold no images.

The competition code lives in this repository. It reuses the FYP's `head_localisation.py` and `head_yaw.py` unchanged (copies in `fyp/`). The robot-side OpenCV 5 backend is in `integration/ros2/`: two new modules and a patch to four FYP files that adds the `yolo_backend` / `opencv5_socket` launch arguments (section 4).

2. Stage 1: the detector in OpenCV 5

Pipeline (`galbot_cv5/yolo_seg_dnn.py`):

1. Ultralytics-compatible letterbox (`cv2.resize`, `cv2.copyMakeBorder`) and `cv2.dnn.blobFromImage`.
2. `cv2.dnn.readNetFromONNX(..., engine=ENGINE_NEW)`.
3. Decoding with class-aware `cv2.dnn.NMSBoxesBatched`.
4. Mask assembly with `cv2.gemm` and `cv2.resize`.
5. Selection of the highest-scoring block, as the robot's adapter does.

Two findings mattered:

- **Input size.** Ultralytics runs `.pt` checkpoints with `rect=True`, so 640×480 images reach the network unpadded at 480×640. A square 640×640 export changed scores by up to 0.059. Exporting at 480×640 made the outputs identical, with raw score differences ≤ 1.3e-6.
- **Thread contention.** NumPy's `@` woke a 16-thread OpenBLAS pool that slowed the next `cv2.dnn` forward pass from about 20 ms to about 70 ms. `cv2.gemm` keeps the work in OpenCV's own thread pool.

Parity and speed. 150 MuJoCo validation images (60 head, 90 wrist; 140 with the block, 10 empty), confidence 0.35, 8 threads, AMD Ryzen 7 8845HS laptop:

	Ultralytics (robot adapter)	**OpenCV 5 `ENGINE_NEW`**	**OpenCV 5 `ENGINE_CLASSIC`**
Detection decisions matching Ultralytics	—	150 / 150	150 / 150
Mask pixels differing (max per image)	—	**0**	**0**
Recall / false positives on empty images	140/140, 0/10	140/140, 0/10	140/140, 0/10
Mask IoU vs ground truth (median)	0.9266	0.9266	0.9266
End-to-end latency, median (p95)	25.3 (30.4) ms	**20.9 (24.8) ms**	51.4 (65.6) ms

Network forward pass only, same tensor, 8 threads: PyTorch 27.0 ms, OpenCV 5 NEW 19.6 ms, CLASSIC 45.5 ms (`results/latency_forward_sweep.jsonl`).

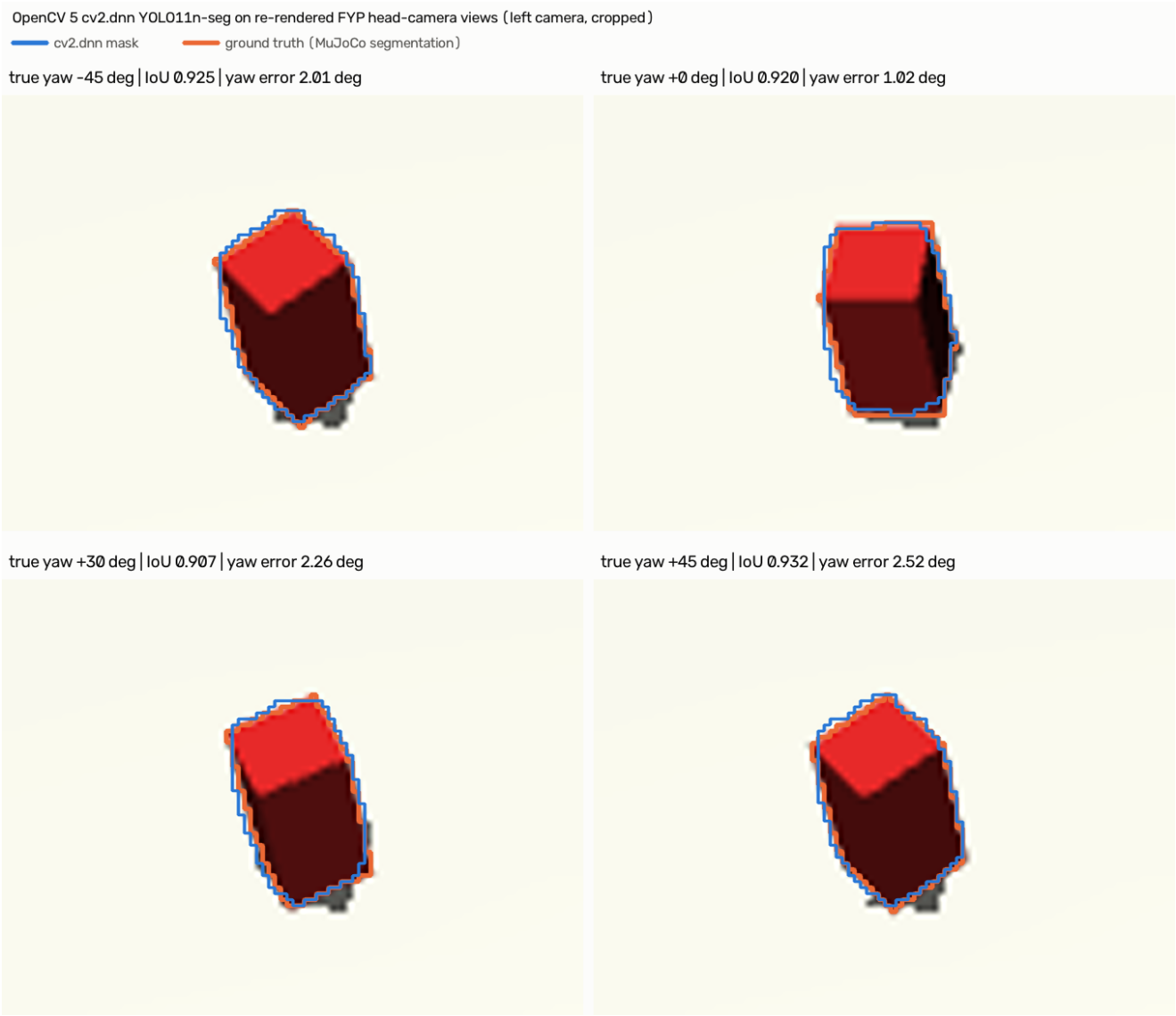
3. Stage 2: the evaluation framework

For each head stereo pair:

1. Get a mask from one of three sources:
    - `dnn`: OpenCV 5, as above.
    - `hsv`: a model-free red threshold.
    - `gt`: MuJoCo segmentation, as an oracle.
2. Run the robot's own `mask_centroid`, `triangulate` and `fit_head_yaw`.
3. Compare with the true block pose. Yaw is compared modulo 90°, because the square footprint only defines yaw up to a quarter turn.

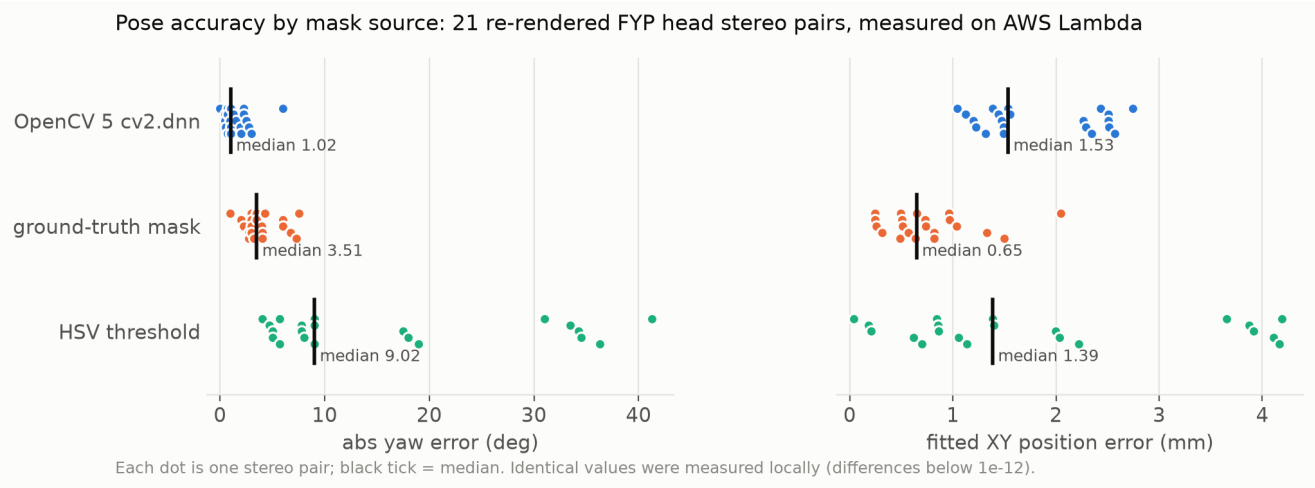
Inputs:

- **Re-rendered robot trials:** 7 trials of the head-yaw sweep (block yaw 0, ±15, ±30, ±45°), 3 samples each, giving 21 stereo pairs. They are rendered from the recorded joints and poses, with ground-truth masks from MuJoCo segmentation.
- **Recorded missions:** 11 missions, scored on the first pose the pick actually used. This is log analysis only.
- **Synthetic:** 12 stereo pairs drawn with `cv2.projectPoints`. They contain no project data.



Results on the 21 re-rendered pairs (median / p95 / max; identical locally and on AWS):

Mask source	Pose found	Mask IoU	Stereo XY error (mm)	Fitted XY error (mm)	Yaw error (deg)
OpenCV 5 dnn	21/21	0.923 / 0.934 / 0.936	1.53 / 2.97 / 4.87	1.53 / 2.57 / 2.74	**1.02 / 2.98 / 6.02**
ground-truth mask	21/21	1.000	0.66 / 1.33 / 1.36	0.65 / 1.50 / 2.05	3.51 / 7.27 / 7.52
HSV threshold	21/21	0.893 / 0.969 / 0.971	6.81 / 10.46 / 10.90	1.39 / 4.16 / 4.19	9.02 / 36.27 / 41.27



Recorded missions (11):

- live XY error 1.62 / 2.49 / 2.99 mm
- yaw error 2.01 / 3.89 / 4.02°
- 10 of 11 missions succeeded; the failure was the +15° headless trial

Synthetic (12 pairs): with ground-truth masks, yaw error is 0.31 / 0.79 / 0.84°. With HSV masks, IoU is 0.984 and yaw error is 0.81 / 2.05 / 2.12°.

Observations:

- **Ground-truth masks give worse yaw than the DNN masks.** MuJoCo's geom segmentation is slightly smaller than the box-model silhouette (IoU about 0.90 at the true pose). On synthetic data, where mask and model agree exactly, ground-truth masks recover yaw within 0.84°.
- **HSV is a weak baseline.** It wasn't tuned to the simulator's lighting, and its yaw tail reaches 41°. That's why the robot moved to learned segmentation.

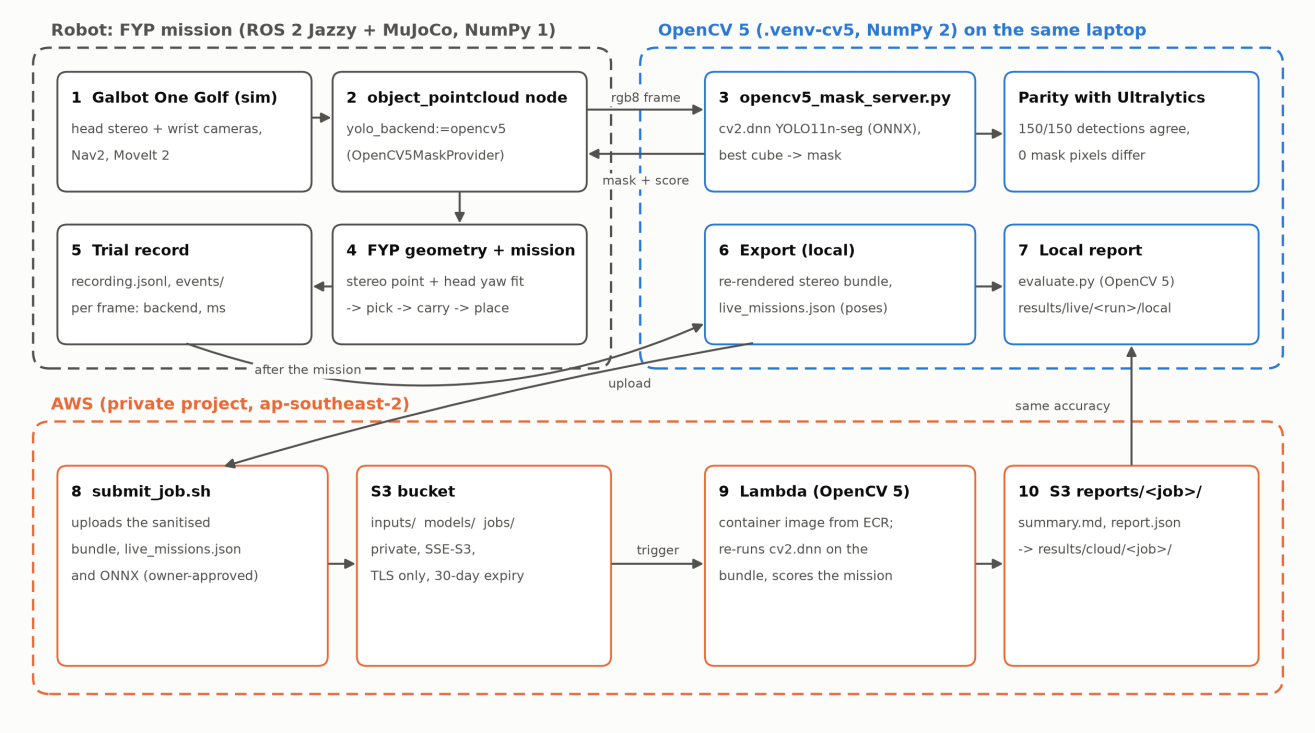
4. Stage 3: OpenCV 5 in the live robot mission

Why a separate process. OpenCV 5 needs NumPy 2, while ROS 2 Jazzy's Python stack uses NumPy 1.26, so they cannot share an interpreter. The detector runs as its own process (`tools/opencv5_mask_server.py`, the same `galbot_cv5` runner as stage 1). The robot's perception node reaches it through `OpenCV5MaskProvider` (`integration/ros2/`) over a local Unix socket:

- fixed `struct` headers (`opencv5_protocol.py`, one file shared by both sides and checked by a test)
- raw `rgb8` frames in; a bit-packed mask, score, forward and total time out
- on a server error the provider rejects that frame and reconnects on the next one; it **never falls back to Ultralytics**, so a mission run with `yolo_backend:=opencv5` only ever used OpenCV 5 masks

The rest of the pipeline is the robot's own: mask geometry, stereo triangulation, silhouette yaw fit, then the Nav2 / MoveIt 2 pick-and-place mission.

How a run is produced and checked. `live/run_live_demo.sh` starts the server, launches the mission with `yolo_backend:=opencv5`, then exports and evaluates the recording (and, with `--cloud`, submits it to AWS). For every frame, the robot records which backend produced the mask; the report lists it per mission (`opencv5_dnn 5.0.0` for all five). With `--record`, the server also saves each frame with the mask it returned; the demo video and GIF are built from those frames next to a replay of the mission.



Results: 5 live missions (block yaw +30°; pick pose = the first pose the pick used, against simulator truth at the same time; source `results/live/`):

Mission	Result	Frames (with block)	Median frame time	Pick XY error	Pick yaw error	Placement error
`171622`	success	268 (268)	34.1 ms	1.60 mm	0.26°	4.6 mm
`173928`	success	154 (154)	42.3 ms	1.71 mm	1.02°	4.2 mm
`180808`	success	170 (162)	46.1 ms	2.28 mm	1.51°	6.1 mm
`203233`	success	242 (242)	33.5 ms	2.86 mm	0.52°	3.3 mm
`203608`	success	252 (252)	35.0 ms	2.55 mm	0.01°	3.6 mm

The frame time is measured by the robot-side provider. It includes the socket round trip and shares the laptop CPU with MuJoCo, Nav2 and MoveIt 2, so it is higher than the 20.9 ms of the isolated benchmark.

Same mission, local and AWS. For missions `173928` and `180808`, the head stereo views were re-rendered from the recording, sanitised and evaluated both locally and on AWS Lambda (stage 2 framework). The accuracy matched exactly:

Mission	dnn mask IoU (median)	Fitted XY error (median)	Yaw error (median)	Local / AWS
`173928`	0.896	2.23 mm	1.27°	identical
`180808`	0.907	1.64 mm	1.76°	identical

5. Reproducible evaluation on AWS

Deployment: one CloudFormation stack in a private AWS project (Sydney Region, Free Plan):

- a private S3 bucket: encrypted, public access blocked, HTTPS only, objects expire after 30 days
- a container-image Lambda: OpenCV 5.0.0, NumPy 2.5.4, x86_64, 3008 MB, 600 s timeout
- an execution role that can only read `inputs/`, `models/` and `jobs/` and write `reports/`, so a report can never re-trigger the function
- a 14-day log group, an error alarm, and a USD 5 monthly budget with email alerts

Trigger: writing `jobs/<id>/job.json` runs the evaluation, and the reports appear under `reports/<id>/`.

Image: pinned by digest, built from a pinned base image. Five base-image libraries (openssl, curl, rpm, pcre2 and libxml2; nine packages) are upgraded from a pinned later Amazon Linux release. The ECR scan went from 12 HIGH and 20 MEDIUM findings to none.

Data hygiene before upload:

- `tools/sanitize_for_upload.py` removes local paths from the bundle metadata and the ONNX `description`
- it checks that the pixel data and weights are unchanged, and writes a SHA-256 manifest
- `aws/submit_job.sh` uploads only synthetic data unless project data is explicitly allowed

Cloud runs:

Job	Data	Lambda duration (billed)	Peak memory	Accuracy vs local
`synthetic-smoke`	12 synthetic pairs, hsv + gt	8.4 s (11.6 s, including a 3.3 s cold start)	190 MB	identical (10 of 736 values differ by about 1e-17)
`fyp-headless-sweep-dnn`	21 robot pairs, dnn + gt + hsv	19.7 s (19.7 s)	298 MB	identical (largest difference 1.2e-13)

```
# Rendered from AWS CLI and CloudWatch Logs output, 2026-10-22. Account ID replaced with <account-id>.

$ aws cloudformation describe-stacks --stack-name galbot-cv5-eval --query Stacks[0].StackStatus
CREATE_COMPLETE    (8 resources: S3 bucket + policy, Lambda + S3 permission, IAM role, logs, alarm, budget)

$ aws s3 ls --recursive s3://galbot-cv5-eval-<account-id>-ap-southeast-2/reports/
2026-10-22 14:26:46      35991 reports/synthetic-smoke/report.json
2026-10-22 14:26:46        874 reports/synthetic-smoke/summary.md
2026-10-22 14:30:14      87778 reports/fyp-headless-sweep-dnn/report.json
2026-10-22 14:30:14       1001 reports/fyp-headless-sweep-dnn/summary.md
2026-10-22 14:28:48        164 reports/fyp-headless-sweep/error.json    # first attempt: model sidecar not uploaded

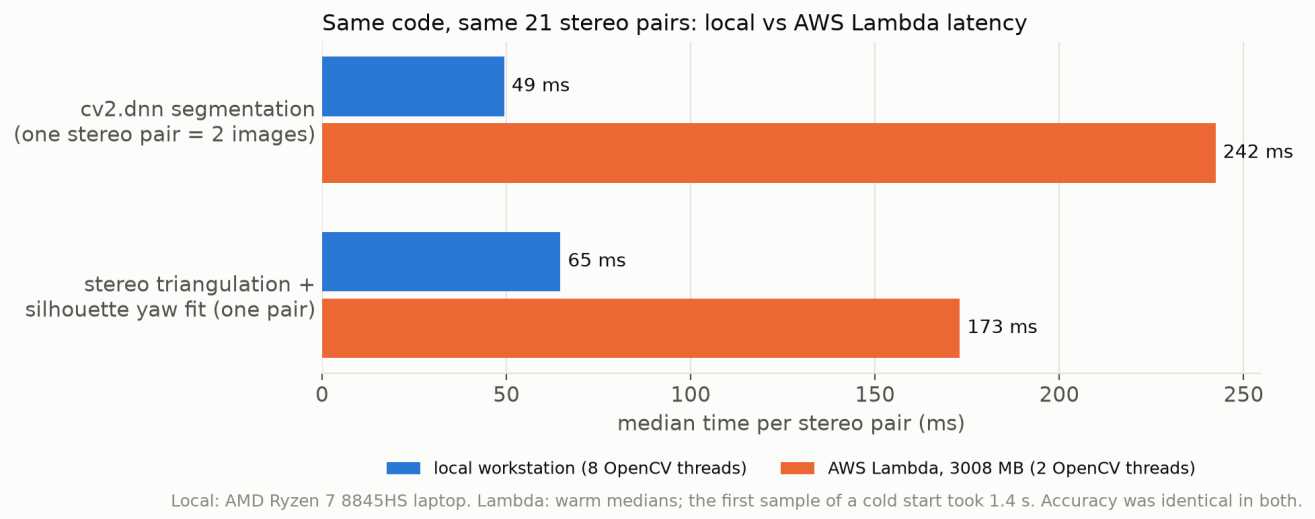
# CloudWatch Logs /aws/lambda/galbot-cv5-eval-evaluator
REPORT  synthetic-smoke         Duration: 8365.56 ms   Billed: 11622 ms  Memory: 3008 MB  Max used: 190 MB  Init: 3256 ms
REPORT  fyp-headless-sweep-dnn  Duration: 19716.13 ms  Billed: 19717 ms  Memory: 3008 MB  Max used: 298 MB

# ECR image scan (opencv5-6af372c): CRITICAL 0  HIGH 0  MEDIUM 0  LOW 0  (1 informational)
# Cost Explorer, month to date: USD 0.00    Free Plan: ACTIVE
```

6. Latency: local and AWS are different measurements

Accuracy carries over exactly; latency does not. The rows below are not interchangeable.

Measurement	Where	CPU / threads	Scope	Median (p95)
OpenCV 5 forward pass only	local	Ryzen 7 8845HS, 8 threads	one 480×640 tensor	19.6 ms
OpenCV 5 end to end	local	same, 8 threads	one image: letterbox to selected mask	**20.9 (24.8) ms**
Ultralytics end to end	local	same, 8 threads	one image (robot adapter)	25.3 (30.4) ms
cv2.dnn segmentation in the evaluation	local	same, 8 threads	one stereo pair (2 images)	49.4 (55.4) ms
cv2.dnn segmentation in the evaluation	**AWS Lambda**	3008 MB, 2 OpenCV threads	one stereo pair (2 images)	**242.4 (246.1) ms**; first sample after a cold start: 1425 ms
triangulation + silhouette yaw fit	local	8 threads	one stereo pair	64.6 (66.7) ms
triangulation + silhouette yaw fit	**AWS Lambda**	2 threads	one stereo pair	173.0 (174.1) ms
OpenCV 5 in the live mission	local	shared with the simulator	one head frame, including the socket round trip	33.5–46.1 ms (per-mission medians)
whole 21-pair job	**AWS Lambda**	3008 MB	download, evaluate, write reports	19.7 s



Lambda gives this function about 2 vCPUs at 3008 MB, against 8 threads on the laptop, so it is about 5× slower for the DNN and about 2.7× slower for the geometry. The cloud is here for **reproducible, isolated, auditable evaluation**, not for real-time inference. Real-time numbers come from the local measurements.

7. Security, cost and operations

- **Scope:** no public endpoints (no function URL, no API), no IAM users and no access keys. Deployment used short-lived `aws login` credentials.
- **Data:** only data the owner approved was uploaded: the synthetic bundle, the sanitised 21-pair bundle, the sanitised bundles of two live missions and the sanitised ONNX model. Recordings and recorded camera frames stay local.
- **Cost:** USD 0.00 month to date in Cost Explorer after the first two runs (not re-checked after the live-mission jobs), and the project is still on the Free Plan. Estimated running cost is about USD 0.03 per month (281 MB in ECR), with Lambda use far inside the free allowance.
- **Teardown:** documented in `aws/README.md`. The resources are being kept until judging ends.

8. Limitations

- **Simulation only.** All images are MuJoCo renders from the same generator as the training set. Nothing here measures real-camera accuracy, and the model is simulation-specific.
- **Small stereo set.** There are 21 re-rendered pairs from 7 trials of one yaw sweep, 11 recorded missions, 5 live OpenCV 5 missions and 12 synthetic pairs. The p95 and max values rest on a few samples, and there are no confidence intervals.
- **Re-rendered, not recorded, frames.** The robot logs store no images, so stage 2 scores re-rendered views of the recorded states. Lighting and timing match only approximately.
- **Parity, not improvement.** OpenCV 5 reproduces the detector exactly. It removes PyTorch from the inference path and is faster locally, but it is no more accurate.
- **Model deployment.** The ONNX graph is static at 480×640. Other sizes are letterboxed, which matches Ultralytics' ONNX behaviour but not its `.pt` behaviour.
- **Live missions cover one case.** All 5 live OpenCV 5 missions used the same block yaw (+30°) and scene. The robot's default backend is still Ultralytics; OpenCV 5 is opt-in.
- **Cloud latency.** It is not representative of on-robot latency, as above.
- **The error alarm doesn't fire for failed jobs.** The handler catches job errors and writes `reports/<id>/error.json`, so Lambda counts the invocation as a success. `aws/submit_job.sh` exits with an error and prints `error.json` in that case. Making the alarm fire would need a template change, which was deferred.
- **Licence.** The weights derive from Ultralytics YOLO11 (AGPL-3.0).

9. Reproduce

The full commands are in `README.md` (environments, benchmark, evaluation, tests), `aws/README.md` (image, change set, jobs) and `live/README.md` (live missions, which need the separate FYP workspace). The figures are regenerated by `tools/figure_overlays.py` (`.venv-cv5`) and `tools/figure_charts.py` (`.venv-export`).